
Wavelet-SRNet in PyTorch

A wavelet-based approach to face super-resolution

Gerardo Lauro

Antonio Maria Improta

Francesco Simone Pirozzi

Master's Degree in Computer Engineering

University of Naples Federico II

Multimedia Signal Processing

Professor Luisa Verdoliva

1 Project overview

This project was developed for the Multimedia Signal Processing course at the University of Naples Federico II. The goal was to study, reimplement and extend *Wavelet-SRNet*, the face super-resolution method proposed by Huang et al. at ICCV 2017 [1].

The original method differs from conventional super-resolution networks because it does not directly predict the high-resolution image. Instead, a convolutional neural network estimates the wavelet coefficients of the target image, which are then used to reconstruct the final high-resolution result. The idea is to work explicitly with low- and high-frequency information, so that both the global structure of the face and its texture details can be represented.

Our implementation focuses on the $8\times$ super-resolution case, where a 16×16 RGB face is reconstructed at 128×128 . The original architecture and loss functions were studied from the paper and reimplemented in PyTorch. We also extended the training objective with an identity loss based on ArcFace embeddings.

The main processing chain can be summarized as

LR face $\rightarrow$ CNN $\rightarrow$ wavelet coefficients
 $\rightarrow$ inverse Haar transform $\rightarrow$ SR face

2 Wavelet representation

2.1 Wavelet packet transform

Wavelet-SRNet is based on the Wavelet Packet Transform (WPT). In a standard wavelet decomposition, an image is separated into low-frequency approximation coefficients and high-frequency detail coefficients. In a full wavelet packet decomposition, this operation is repeated on every resulting band.

The original paper uses the Haar wavelet because of its simple structure. For a one-dimensional signal, the low-pass and high-pass Haar filters are

$$h_{\text{low}} = \frac{1}{\sqrt{2}}(1, 1), \quad h_{\text{high}} = \frac{1}{\sqrt{2}}(1, -1). \quad (1)$$

Applying the filters along rows and columns produces four sub-bands at each decomposition step. Repeating this process gives a multi-level wavelet packet representation.

For an n -level decomposition, the scale factor r and the number of wavelet coefficients N_w are [1]

$$r = 2^n, \quad N_w = 4^n. \quad (2)$$

For the $8\times$ configuration used in the project, $n = 3$, therefore

$$r = 8, \quad N_w = 64. \quad (3)$$

Each coefficient has the same spatial size as the 16×16 low-resolution input. Since each coefficient contains three RGB channels, the network predicts $64\times 3 = 192$ channels in total.

3 Wavelet-SRNet

3.1 Super-resolution in the wavelet domain

A conventional super-resolution network learns a mapping from a low-resolution image x to an estimate $\hat{y}$ of the high-resolution image y . A common objective is the mean squared error

$$l_{\text{mse}}(\hat{y}, y) = \|\hat{y} - y\|_F^2. \quad (4)$$

As discussed in the original paper, optimizing only an image-space MSE can lead to smooth results and loss of high-frequency details. Wavelet-SRNet therefore moves the prediction problem to the wavelet domain.

Let

$$C = (c_1, c_2, \dots, c_{N_w}), \quad \hat{C} = (\hat{c}_1, \hat{c}_2, \dots, \hat{c}_{N_w}) \quad (5)$$

be the target and predicted wavelet coefficients. The CNN learns to estimate $\hat{C}$ from the low-resolution input. The final super-resolved image is obtained by reconstructing these coefficients with the inverse Haar transform.

3.2 Network architecture

The network follows the main structure described by Huang et al. and can be divided into three logical stages:

- **Embedding:** the low-resolution RGB image is converted into a set of feature maps.
- **Wavelet prediction:** the extracted features are used to predict the wavelet coefficients required for reconstruction.
- **Reconstruction:** the predicted coefficients are transformed back to image space to obtain the high-resolution face.

The convolutional part keeps the same spatial resolution as the input. In our $8\times$ configuration, the network receives a tensor of size $3 \times 16 \times 16$ and predicts $192 \times 16 \times 16$ wavelet channels. A fixed inverse Haar transform then reconstructs the $3 \times 128 \times 128$ output image.

The project was implemented in PyTorch, replacing the Caffe implementation used in the original work. The code is divided into separate modules for the network, wavelet operations, losses, data loading, training and evaluation.

4 Training objective

The original Wavelet-SRNet combines losses in the wavelet domain with a loss on the reconstructed image.

4.1 Wavelet prediction loss

The wavelet prediction loss is a weighted MSE over the predicted coefficients:

$$l_{\text{wavelet}}(C, \hat{C}) = \sum_{i=1}^{N_w} \lambda_i \|\hat{c}_i - c_i\|_F^2. \quad (6)$$

The weights λ_i control the importance of the different bands. The approximation coefficient mainly describes the global structure, while the higher-frequency coefficients contain finer details.

4.2 Texture loss

The paper also introduces a texture loss to discourage high-frequency coefficients from becoming too small:

$$l_{\text{texture}} = \sum_{i=k}^{N_w} \gamma_i \max \left(\alpha \|c_i\|_F^2 + \epsilon - \|\hat{c}_i\|_F^2, 0 \right). \quad (7)$$

This term is intended to preserve texture information that could otherwise be lost during training.

4.3 Full-image loss

After reconstruction, an additional MSE is computed in image space. The complete objective proposed in the original paper is

$$\begin{aligned} l_{\text{total}} = & \sum_{i=1}^{N_w} \lambda_i \|\hat{c}_i - c_i\|_F^2 \\ & + \mu \sum_{i=k}^{N_w} \gamma_i \max \left(\alpha \|c_i\|_F^2 + \epsilon - \|\hat{c}_i\|_F^2, 0 \right) \\ & + \nu \|R\hat{C} - y\|_F^2, \end{aligned} \quad (8)$$

where R represents the reconstruction operation and $R\hat{C} = \hat{y}$.

4.4 Identity loss extension

As an extension to the original method, the project adds an identity loss based on a pretrained ArcFace model [2]. The reconstructed and ground-truth faces are passed through the frozen face-recognition network and their embeddings are compared using cosine distance. If $f(\cdot)$ denotes the ArcFace embedding, the identity term is

$$L_{\text{ID}} = \frac{1}{B} \sum_{b=1}^B \left(1 - \frac{f(\hat{y}_b)^T f(y_b)}{\|f(\hat{y}_b)\|_2 \|f(y_b)\|_2} \right). \quad (9)$$

The final loss used in our implementation combines the original reconstruction terms with the identity loss:

$$L = 0.01L_{\text{low}} + L_{\text{high}} + L_{\text{tex}} + 0.1L_{\text{img}} + 0.03L_{\text{ID}}. \quad (10)$$

The purpose of the additional identity term is to encourage the super-resolved image to preserve identity-related features. The ArcFace network remains frozen during training, so only the super-resolution network is updated.

5 Dataset and training

The project uses aligned face images from the CelebA dataset. High-resolution images are prepared at 128×128 , while the corresponding low-resolution inputs are obtained by bicubic downsampling to 16×16 .

The PyTorch training pipeline includes data loading, checkpoint saving and validation. The model is trained with Adam and a cosine learning-rate schedule. Mixed-precision training is supported to reduce GPU memory usage and improve training speed.

For evaluation, the project includes common super-resolution metrics such as PSNR and SSIM, together with LPIPS for perceptual similarity. ArcFace embeddings can also be used to compare identity similarity between reconstructed and reference faces.

5.1 Cross-Dataset Evaluation on Helen

To evaluate the model on images outside the training dataset, we also tested it on 100 faces from the Helen dataset. No fine-tuning was performed on Helen, so this experiment was used as a simple cross-dataset test of the model’s generalization.

The same $8\times$ setting used for CelebA was considered, with 16×16 low-resolution inputs reconstructed at 128×128 . The results are reported below.

Method	PSNR $\uparrow$	SSIM $\uparrow$	ArcFace $\uparrow$
Bicubic	21.96	0.617	0.218
Wavelet-SRNet baseline	21.92	0.509	0.262
Wavelet-SRNet + Identity Loss	22.13	0.564	0.285

Adding the Identity Loss improves the Wavelet-SRNet baseline on all three metrics on Helen. In particular, the ArcFace cosine similarity increases from 0.262 to 0.285, while PSNR and SSIM also improve. Bicubic interpolation still obtains a higher SSIM, but its ArcFace similarity is lower than both learned models.

Since Helen is used only for testing and is not seen during training, this experiment provides an additional check of how the model behaves on a different face dataset.

6 Software organization

The implementation is organized into small modules with separate responsibilities:

Module	Responsibility
<code>models/waveletsrnet.py</code>	Wavelet-SRNet architecture and Haar reconstruction
<code>models/arcface.py</code>	Pretrained ArcFace model used for identity features
<code>losses/</code>	Wavelet, texture, image and identity losses
<code>dataset.py</code>	Dataset loading and LR/HR image preparation
<code>train.py</code>	Training loop, validation and checkpoints
<code>metrics/</code>	Image-quality and perceptual evaluation metrics
<code>tools/</code>	Evaluation, profiling and utility scripts

The modular structure made it easier to compare the implementation with the original paper, test individual components and extend the training objective without changing the main network pipeline.

References

- [1] H. Huang, R. He, Z. Sun and T. Tan, “Wavelet-SRNet: A Wavelet-Based CNN for Multi-Scale Face Super Resolution,” *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [2] J. Deng, J. Guo, N. Xue and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.